

SemiGuard AI - NotebookLM 전문가 가이드

빠른 시작

이 문서의 목적

Manus 크레딧이 없을 때 NotebookLM을 사용하여 프로젝트를 계속 진행하기 위한 완벽한 가이드입니다.

NotebookLM에 업로드할 파일

1. 이 파일 (NOTEBOOKLM_GUIDE.md)
 2. 프로젝트의 주요 소스 코드 (또는 코드 스니펫)
 3. GitHub 저장소 링크
-

프로젝트 개요

프로젝트명

SemiGuard AI - 반도체 장비 예지안전 시스템

목표

- 반도체 장비의 센서 데이터를 실시간으로 분석
- AI를 활용하여 이상 상태를 미리 탐지
- 사용자별 데이터 격리 및 공유 기능 제공

기술 스택

영역	기술
프론트엔드	React 19, Tailwind CSS 4, TypeScript
백엔드	Express 4, Node.js, tRPC 11
데이터베이스	MySQL / TiDB
인증	OAuth 2.0 (Google, Naver, Kakao, Manus)
배포	Manus (Cloud Run)

현재 배포 URL

- 프로덕션: <https://semiguardai-jifnzsvd.manus.space>
 - 개발 서버: <http://localhost:3000>
-

프로젝트 아키텍처

폴더 구조

```
semiguard-ai/
├── client/                                     # React 프론트엔드
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Login.tsx                    # 로그인 페이지 (Google, Naver, Kakao)
│   │   │   ├── Dashboard.tsx               # 메인 대시보드
│   │   │   └── NotFound.tsx                # 404 페이지
│   │   ├── components/
│   │   │   ├── DashboardLayout.tsx         # 대시보드 레이아웃
│   │   │   ├── AIChatBox.tsx              # AI 채팅 인터페이스
│   │   │   └── ui/                         # shadcn/ui 컴포넌트
│   │   ├── _core/
│   │   │   └── hooks/
│   │   │       └── useAuth.ts              # 인증 상태 관리
│   │   ├── lib/
│   │   │   ├── trpc.ts                    # tRPC 클라이언트 설정
│   │   │   └── utils.ts                   # 유틸리티 함수
│   │   ├── const.ts                      # 상수 및 로그인 함수
│   │   ├── App.tsx                       # 메인 라우터
│   │   └── main.tsx                      # 엔트리 포인트
│   └── public/
│       └── favicon.ico
├── server/                                    # Express 백엔드
│   ├── _core/
│   │   ├── index.ts                      # 서버 메인 파일
│   │   ├── context.ts                   # tRPC 컨텍스트
│   │   ├── trpc.ts                      # tRPC 설정
│   │   ├── oauth.ts                     # Manus OAuth
│   │   ├── socialOAuth.ts               # 소셜 OAuth (Google, Naver, Kakao)
│   │   ├── cookies.ts                   # 쿠키 설정
│   │   ├── env.ts                       # 환경 변수
│   │   ├── llm.ts                       # LLM 통합
│   │   ├── imageGeneration.ts           # 이미지 생성
│   │   └── sdk.ts                       # Manus SDK
│   ├── routers.ts                       # 모든 tRPC 라우터
│   ├── db.ts                            # 데이터베이스 쿼리
│   └── storage.ts                       # S3 스토리지
```

├── semiguard.ts	# 데이터 분석 로직
├── semiguardDb.ts	# SemiGuard DB 함수
└── auth.logout.test.ts	# 테스트
├── drizzle/	# 데이터베이스 스키마
│ ├── schema.ts	# 테이블 정의
│ ├── relations.ts	# 테이블 관계
│ └── migrations/	# 마이그레이션 파일
├── shared/	# 공유 코드
│ ├── const.ts	# 공유 상수
│ ├── types.ts	# 공유 타입
│ └── semiguard.ts	# 분석 로직
└── package.json	# 의존성

데이터 흐름

1. 사용자 로그인
 - ├── 소셜 로그인 (Google/Naver/Kakao)
 - ├── OAuth 콜백 처리
 - └── 세션 쿠키 생성
2. 데이터 분석
 - ├── 센서 데이터 입력
 - ├── AI 모델 분석
 - └── 이상 탐지 결과 저장
3. 대시보드 표시
 - ├── 사용자별 데이터 조회
 - ├── 실시간 모니터링
 - └── 통계 및 리포트

데이터베이스 스키마

users 테이블

```
CREATE TABLE users (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  openId VARCHAR(255) UNIQUE NOT NULL,      -- OAuth 제공자 ID  
  email VARCHAR(255),  
  name VARCHAR(255),  
  provider VARCHAR(50),                    -- 'google', 'naver', 'kakao'  
  role ENUM('user', 'admin') DEFAULT 'user',  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

anomaly_logs 테이블

```
CREATE TABLE anomaly_logs (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  userId INT NOT NULL,  
  timestamp BIGINT,                        -- Unix 타임스탬프 (ms)  
  current DECIMAL(10, 2),                  -- 전류 (A)  
  temperature DECIMAL(10, 2),              -- 온도 (°C)  
  vibration DECIMAL(10, 2),                -- 진동 (mm/s)  
  noise DECIMAL(10, 2),                    -- 소음 (dB)  
  anomalyScore DECIMAL(10, 2),             -- 이상 점수 (0-100)  
  riskLevel VARCHAR(20),                   -- 'normal', 'caution', 'warn:  
  isAnomaly BOOLEAN,  
  llmAnalysis TEXT,                        -- LLM 분석 결과  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (userId) REFERENCES users(id),  
  INDEX idx_userId_timestamp (userId, timestamp)  
);
```

data_sharing 테이블

```
CREATE TABLE data_sharing (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  ownerId INT NOT NULL,           -- 데이터 소유자  
  sharedWithId INT NOT NULL,      -- 공유 대상  
  permission VARCHAR(50) DEFAULT 'read', -- 'read', 'write', 'admin'  
  createdAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (ownerId) REFERENCES users(id),  
  FOREIGN KEY (sharedWithId) REFERENCES users(id),  
  UNIQUE KEY unique_sharing (ownerId, sharedWithId)  
);
```

주요 인덱스

- `users.openId` - OAuth 로그인 빠른 조회
- `anomaly_logs.userId` - 사용자별 데이터 조회
- `anomaly_logs.timestamp` - 시간 범위 조회
- `data_sharing.ownerId` - 공유 데이터 조회

API 엔드포인트 (tRPC 라우터)

인증 (auth)

```
// 현재 사용자 정보 조회  
trpc.auth.me.useQuery()  
// 반환: User | null  
  
// 로그아웃  
trpc.auth.logout.useMutation()  
// 반환: { success: true }
```

SemiGuard 분석 (semiguard)

```
// 정상 데이터 주입
trpc.semiguard.injectNormal.useMutation()
// 입력: { current, temperature, vibration, noise }
// 반환: { anomalyScore, riskLevel, isAnomaly }

// 이상 데이터 주입
trpc.semiguard.injectAnomaly.useMutation()

// 최근 이상 로그 조회
trpc.semiguard.getRecentAnomalyLogs.useQuery(limit)
// 반환: AnomalyLogEntry[]

// 임계값 조회
trpc.semiguard.getThresholds.useQuery()
// 반환: { normal: 29, caution: 49, warning: 69 }

// 임계값 저장
trpc.semiguard.saveThresholds.useMutation()
// 입력: { normal, caution, warning }
```

✅ 완료된 작업

1. 소셜 로그인 구현 ✅

파일: /server/_core/socialOAuth.ts

구현 내용:

- Google OAuth 콜백 처리
- Naver OAuth 콜백 처리
- Kakao OAuth 콜백 처리
- 사용자 정보 DB 저장

클라이언트 함수 (/client/src/const.ts):

```
startGoogleLogin()    // Google 로그인 시작
startNaverLogin()      // Naver 로그인 시작
startKakaoLogin()      // Kakao 로그인 시작
startLogin()           // Manus 로그인 시작
```

2. 로그인 페이지 UI

파일: /client/src/pages/Login.tsx

특징:

- 다크 테마 디자인
- 4개 소셜 로그인 버튼
- 반응형 레이아웃
- 로딩 상태 처리

3. 환경 변수 설정

설정된 변수:

```
VITE_GOOGLE_CLIENT_ID=778659377051-3smspbatf9d3osp8o8rujpgbse6gp5e1.apps.d
VITE_NAVER_CLIENT_ID=qP16ddzamZ9zVFamdpx4
VITE_KAKAO_CLIENT_ID=4f71042a9104e77575f04d93eb01882d
```

4. 서버 포트 설정 수정

파일: /server/_core/index.ts

변경 사항:

- 동적 포트 선택 제거
 - 고정 포트 3000 사용
 - 프론트엔드-백엔드 연결 안정화
-

! 진행중인 작업

인증 기반 라우팅 !

파일: /client/src/App.tsx

현재 상황:

- ProtectedRoute 컴포넌트 구현 중
- 로그인 안 된 사용자를 /login 으로 리다이렉트하려고 함

문제:

- 로그인 안 된 상태에서도 대시보드가 보임
- useAuth() 혹은 loading 상태가 제대로 업데이트되지 않음

해결 방법:

```
// App.tsx의 ProtectedRoute 컴포넌트
function ProtectedRoute({ children }: { children: React.ReactNode }) {
  const { user, loading } = useAuth();
  const [, setLocation] = useLocation();

  useEffect(() => {
    if (!loading && !user) {
      setLocation("/login"); // 로그인 페이지로 리다이렉트
    }
  }, [user, loading, setLocation]);

  if (loading) {
    return <LoadingSpinner />;
  }

  if (!user) {
    return null;
  }

  return <>{children}</>;
}
```

■ 남은 작업 (우선순위)

1. 데이터 격리 (높음) ■

목표: 사용자별 데이터만 조회

작업:

- `protectedProcedure` 로 모든 쿼리 변환
- `ctx.user.id` 기반으로 WHERE 절 추가
- 예: `WHERE userId = ctx.user.id`

영향받는 파일:

- `/server/routers.ts` - 모든 쿼리 수정
- `/server/semiguardDb.ts` - DB 함수 수정

2. 데이터 공유 기능 (중간) ■

목표: 사용자가 다른 사용자와 데이터 공유

작업:

- `data_sharing` 테이블 활용
- 공유 권한 검증 로직
- UI에 공유 버튼 추가

3. 사용자 프로필 페이지 (중간) ■

목표: 사용자 정보 관리

작업:

- `/profile` 라우트 추가
- 프로필 수정 UI
- 로그아웃 버튼

4. 테스트 및 배포 (낮음) ■

작업:

- vitest로 모든 라우터 테스트
- 배포 테스트

- 프로덕션 환경 확인



현재 이슈 및 해결 방법

이슈 1: 로그인 안 된 상태에서 대시보드 접근

증상: / 경로에 접속하면 로그인 페이지 대신 대시보드가 보임

원인: `useAuth()` 혹은 `loading` 상태가 제대로 작동하지 않음

해결 방법:

1. `App.tsx` 의 `ProtectedRoute` 컴포넌트 확인
2. `useAuth()` 혹은 `meQuery.isLoading` 상태 확인
3. tRPC 요청이 제대로 응답하는지 확인

테스트 방법:

```
// 브라우저 콘솔에서 실행
localStorage.removeItem('manus-runtime-user-info');
sessionStorage.removeItem('manus-cookie');
window.location.href = '/';
// 로그인 페이지가 보여야 함
```

이슈 2: 환경 변수 설정 확인

확인 사항:

- Google, Naver, Kakao 클라이언트 ID 설정 여부
- 환경 변수가 클라이언트에 전달되는지 확인

확인 방법:

```
// 브라우저 콘솔에서 실행
console.log(import.meta.env.VITE_GOOGLE_CLIENT_ID);
console.log(import.meta.env.VITE_NAVER_CLIENT_ID);
console.log(import.meta.env.VITE_KAKAO_CLIENT_ID);
```



NotebookLM에 질문하는 방법

효과적인 질문 방식

1 현재 상황 설명 후 질문

"SemiGuard AI 프로젝트는 React + Express + tRPC 스택입니다.
현재 로그인 안 된 상태에서도 대시보드가 보이는 문제가 있습니다.

App.tsx에서 ProtectedRoute 컴포넌트를 만들었는데,
왜 `useAuth()`의 loading 상태가 제대로 작동하지 않을까요?

해결 방법을 단계별로 설명해 주세요."

2 코드 리뷰 요청

"다음 코드를 리뷰해 주세요. 문제점이 있을까요?"

[코드 붙여넣기]

특히 다음 부분이 걱정됩니다:

- `useEffect` 의존성 배열
- 로딩 상태 처리
- 리다이렉트 로직"

3 다음 단계 제안 요청

"소셜 로그인에 완료되었습니다.
다음으로 데이터 격리를 구현하려고 합니다.

현재 모든 쿼리가 전역 데이터를 반환합니다.
userId 기반으로 데이터를 필터링하려면 어떻게 해야 할까요?

단계별 구현 방법을 알려 주세요."

4 에러 해결 요청

"다음 에러가 발생했습니다:

[에러 메시지]

이 에러의 원인이 될까요?
어떻게 해결할 수 있을까요?"

5 아키텍처 설계 상담

"데이터 공유 기능을 구현하려고 합니다.

요구사항:

- 사용자 A가 사용자 B와 데이터 공유
- 읽기/쓰기 권한 구분
- 공유 취소 기능

이를 위해:

1. 데이터베이스 스키마는 어떻게 설계할까요?
2. 백엔드 API는 어떻게 구현할까요?
3. 프론트엔드 UI는 어떻게 만들까요?"

주요 코드 스니펫

소셜 로그인 플로우

파일: /client/src/const.ts

```
export const startGoogleLogin = () => {
  const clientId = import.meta.env.VITE_GOOGLE_CLIENT_ID;
  const redirectUri = `${window.location.origin}/api/oauth/google/callback`;
  const url = new URL("https://accounts.google.com/o/oauth2/v2/auth");
  url.searchParams.set("client_id", clientId);
  url.searchParams.set("redirect_uri", redirectUri);
  url.searchParams.set("response_type", "code");
  url.searchParams.set("scope", "openid email profile");
  window.location.href = url.toString();
};
```

인증 후

파일: /client/src/_core/hooks/useAuth.ts

```
export function useAuth(options?: UseAuthOptions) {
  const meQuery = trpc.auth.me.useQuery(undefined, {
    retry: false,
    refetchOnWindowFocus: false,
  });

  const state = useMemo(() => {
    return {
      user: meQuery.data ?? null,
      loading: meQuery.isLoading,
      isAuthenticated: Boolean(meQuery.data),
    };
  }, [meQuery.data, meQuery.isLoading]);

  return state;
}
```

tRPC 라우터

파일: /server/routers.ts

```
export const appRouter = router({
  auth: router({
    me: publicProcedure.query(opts => opts.ctx.user),
    logout: publicProcedure.mutation(({ ctx }) => {
      ctx.res.clearCookie(COOKIE_NAME);
      return { success: true };
    }),
  }),
  semiguard: router({
    injectNormal: publicProcedure.mutation(async () => {
      // 데이터 분석 로직
    }),
  }),
});
```

유용한 링크

GitHub 저장소

- 저장소: [https://github.com/\[username\]/semiguard-ai](https://github.com/[username]/semiguard-ai)
- 이슈: GitHub Issues에서 문제 추적
- 커밋: 진행 상황을 커밋 메시지로 기록

배포

- 프로덕션: <https://semiguardai-jifnzsvd.manus.space>
- 개발: <http://localhost:3000>

문서

- README.md: 프로젝트 개요
- PROGRESS.md: 진행 상황
- 이 파일: NotebookLM 가이드

NotebookLM 사용 체크리스트

파일 준비

- ☐ 이 파일 (NOTEBOOKLM_GUIDE.md) 준비
- ☐ GitHub 저장소 링크 준비
- ☐ 주요 소스 코드 복사

NotebookLM 설정

- ☐ NotebookLM.com 접속
- ☐ 새 Notebook 생성
- ☐ 이 파일 업로드
- ☐ GitHub 저장소 링크 추가

질문 시작

- ☐ 현재 상황 설명
- ☐ 구체적인 질문 작성
- ☐ 코드 또는 에러 메시지 포함
- ☐ 원하는 결과 명시

NotebookLM 팁

효과적인 질문 작성

- 컨텍스트 제공:** “SemiGuard AI는 반도체 장비 모니터링 시스템입니다”
- 구체적인 문제:** “로그인 안 된 상태에서도 대시보드가 보입니다”
- 시도한 것:** “ProtectedRoute 컴포넌트를 만들었습니다”
- 원하는 결과:** “로그인 페이지로 리다이렉트되어야 합니다”

좋은 예

"SemiGuard AI는 React + Express + tRPC 스택의 반도체 모니터링 시스템입니다.

현재 문제:

- 로그인 안 된 사용자가 대시보드에 접근할 수 있습니다
- `useAuth()` 혹은 `loading` 상태가 제대로 작동하지 않습니다

시도한 것:

- `App.tsx`에서 `ProtectedRoute` 컴포넌트 구현
- `useAuth()` 혹은 `loading` 상태 확인

원하는 결과:

- 로그인 안 된 사용자는 `/login`으로 자동 리다이렉트
- 로딩 중에는 스피너 표시

어떻게 해결할 수 있을까요?"

나쁜 예

"로그인이 안 돼요. 어떻게 해야 해요?"

☎ 추가 지원

문제 해결 순서

1. 이 문서 읽기
2. GitHub Issues 확인
3. NotebookLM에 질문
4. 코드 수정 및 테스트
5. GitHub에 커밋

기록 남기기

```
# 진행 상황 업데이트  
git add .  
git commit -m "NotebookLM 가이드 추가"  
git push origin main
```

마지막 업데이트: 2026년 8월 6일 **작성자:** Manus AI **버전:** 1.0